In [1]:

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
<IPython.core.display.HTML object>
```

stdout:

```
Saving EDA.csv to EDA.csv
```

In [3]:

```
import pandas as pd
df = pd.read_csv(r'EDA.csv')
df
```

Out [3]:

	Customer_ID	Age	Income	Spending_Score	Gender	Purchased
0	1	56	25287	18	Male	0
1	2	46	54387	40	Female	1
2	3	32	28512	21	Female	0
3	4	60	21342	15	Female	0
4	5	25	83076	97	Male	0
...	...	...	...	...	...	...
995	996	22	37778	70	Male	1
996	997	40	65241	44	Male	1
997	998	27	62229	73	Female	1
998	999	61	106652	45	Female	0
999	1000	19	68364	56	Male	0

1000 rows × 6 columns

```
     Customer_ID  Age  Income  Spending_Score  Gender  Purchased
0              1   56   25287              18    Male          0
1              2   46   54387              40  Female          1
2              3   32   28512              21  Female          0
3              4   60   21342              15  Female          0
4              5   25   83076              97    Male          0
..           ...  ...     ...             ...     ...        ...
995          996   22   37778              70    Male          1
996          997   40   65241              44    Male          1
997          998   27   62229              73  Female          1
998          999   61  106652              45  Female          0
999         1000   19   68364              56    Male          0

[1000 rows x 6 columns]
```

In [4]:

```
# 1. What are the dimensions of the dataset (rows and columns)?


df.shape
```

Out [4]:

```
(1000, 6)
```

In [5]:

```
# 2. What are the data types of each column?


df.dtypes
```

Out [5]:

	0
Customer_ID	int64
Age	int64
Income	int64
Spending_Score	int64
Gender	object
Purchased	int64

dtype: object

```
Customer_ID        int64
Age                int64
Income             int64
Spending_Score     int64
Gender            object
Purchased          int64
dtype: object
```

In [6]:

```
# 3. Are there any missing values in the dataset?


df.isnull().sum()
```

Out [6]:

	0
Customer_ID	0
Age	0
Income	0
Spending_Score	0
Gender	0
Purchased	0

dtype: int64

```
Customer_ID       0
Age               0
Income            0
Spending_Score    0
Gender            0
Purchased         0
dtype: int64
```

In [7]:

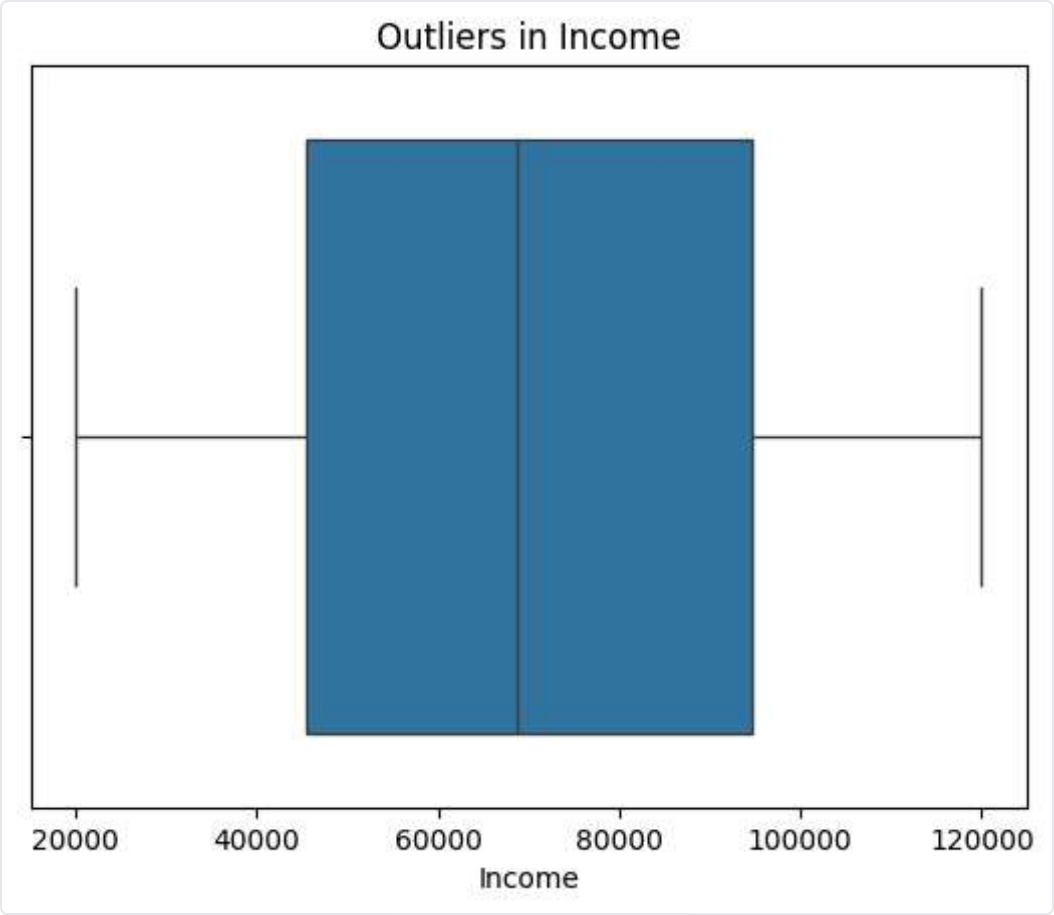
```
# 4. Check for duplicate records. How many exist?


df.duplicated().sum()
```

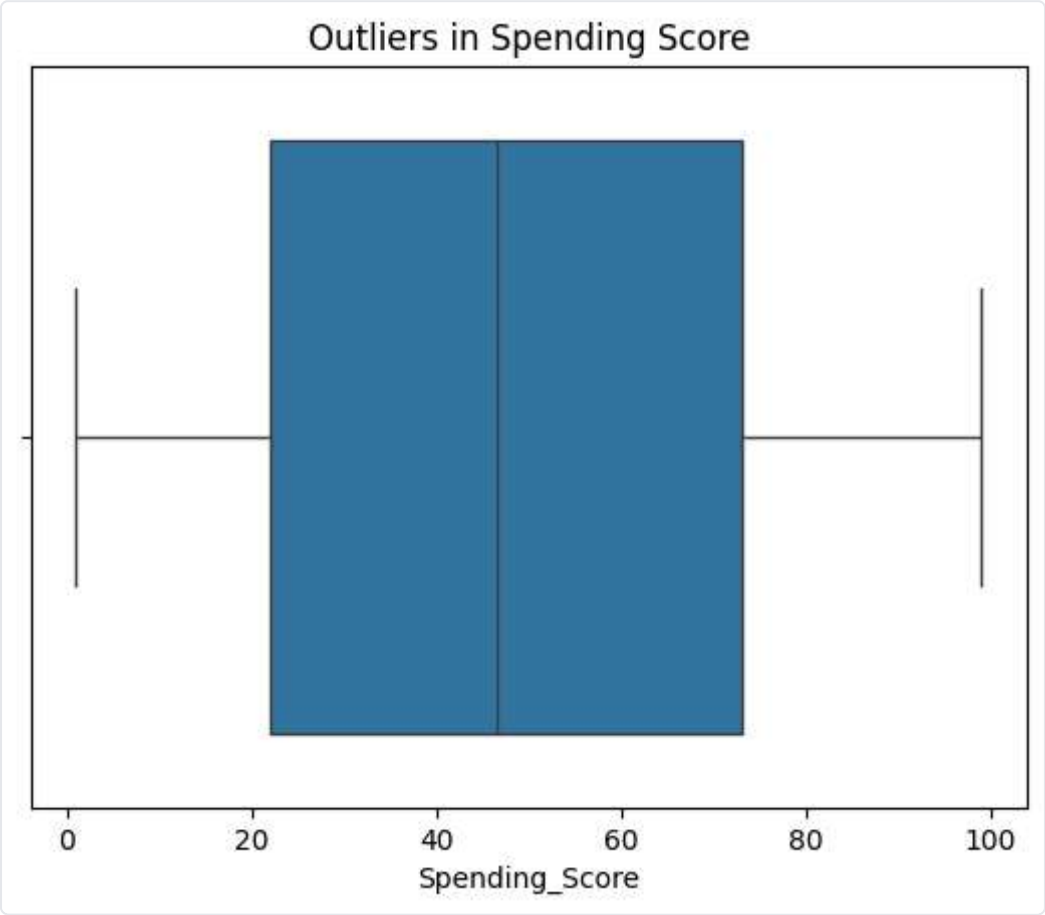
Out [7]:

```
np.int64(0)
```

In [11]:

```
# 5. Are there any outliers in Income or Spending Score?

import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x=df['Income'])
plt.title("Outliers in Income")
plt.show()

sns.boxplot(x=df['Spending_Score'])
plt.title("Outliers in Spending Score")
plt.show()
```


Outliers in Income
20000
40000
60000
80000
100000
120000
Income


Outliers in Spending Score
0
20
40
60
80
100
Spending_Score

In [16]:

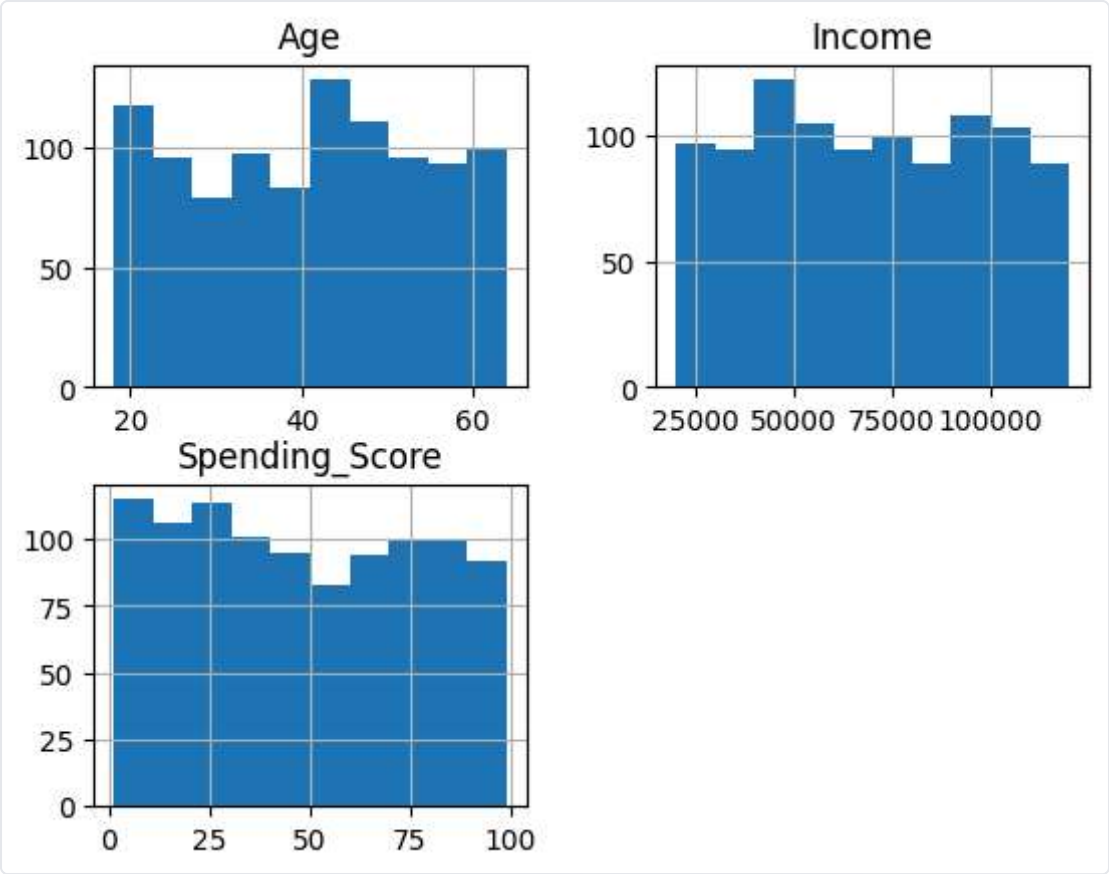
```
# 6. Handle missing values using mean, median, or mode.

df['Income'] = df['Income'].fillna(df['Income'].mean())
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

print(df)
```

stdout:

```
     Customer_ID  Age  Income  Spending_Score  Gender  Purchased
0              1   56   25287              18    Male          0
1              2   46   54387              40  Female          1
2              3   32   28512              21  Female          0
3              4   60   21342              15  Female          0
4              5   25   83076              97    Male          0
..           ...  ...     ...             ...     ...        ...
995          996   22   37778              70    Male          1
996          997   40   65241              44    Male          1
997          998   27   62229              73  Female          1
998          999   61  106652              45  Female          0
999         1000   19   68364              56    Male          0

[1000 rows x 6 columns]
```

In [18]:

```
# 7. What is the distribution of Age, Income, and Spending Score?

df[['Age','Income','Spending_Score']].hist()
plt.show()
```



In [25]:

```
# 8. What is the proportion of customers by Gender?


df['Gender'].value_counts()
```

Out [25]:

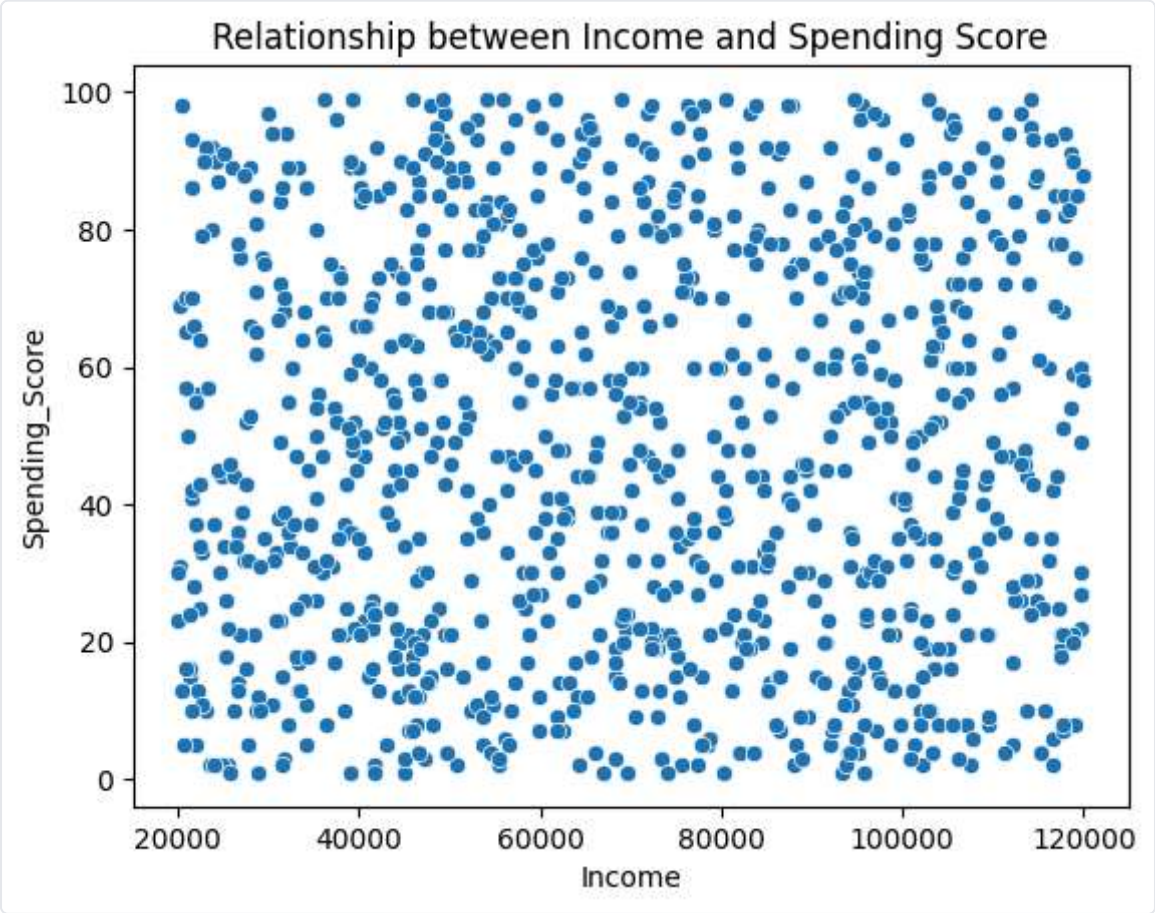
	count
Gender	
Male	518
Female	482

dtype: int64

```
Gender
Male      518
Female    482
Name: count, dtype: int64
```

In [26]:

```
# 9. Analyze relationship between Income and Spending Score.

import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(x='Income', y='Spending_Score', data=df)
plt.title("Relationship between Income and Spending Score")
plt.show()
```



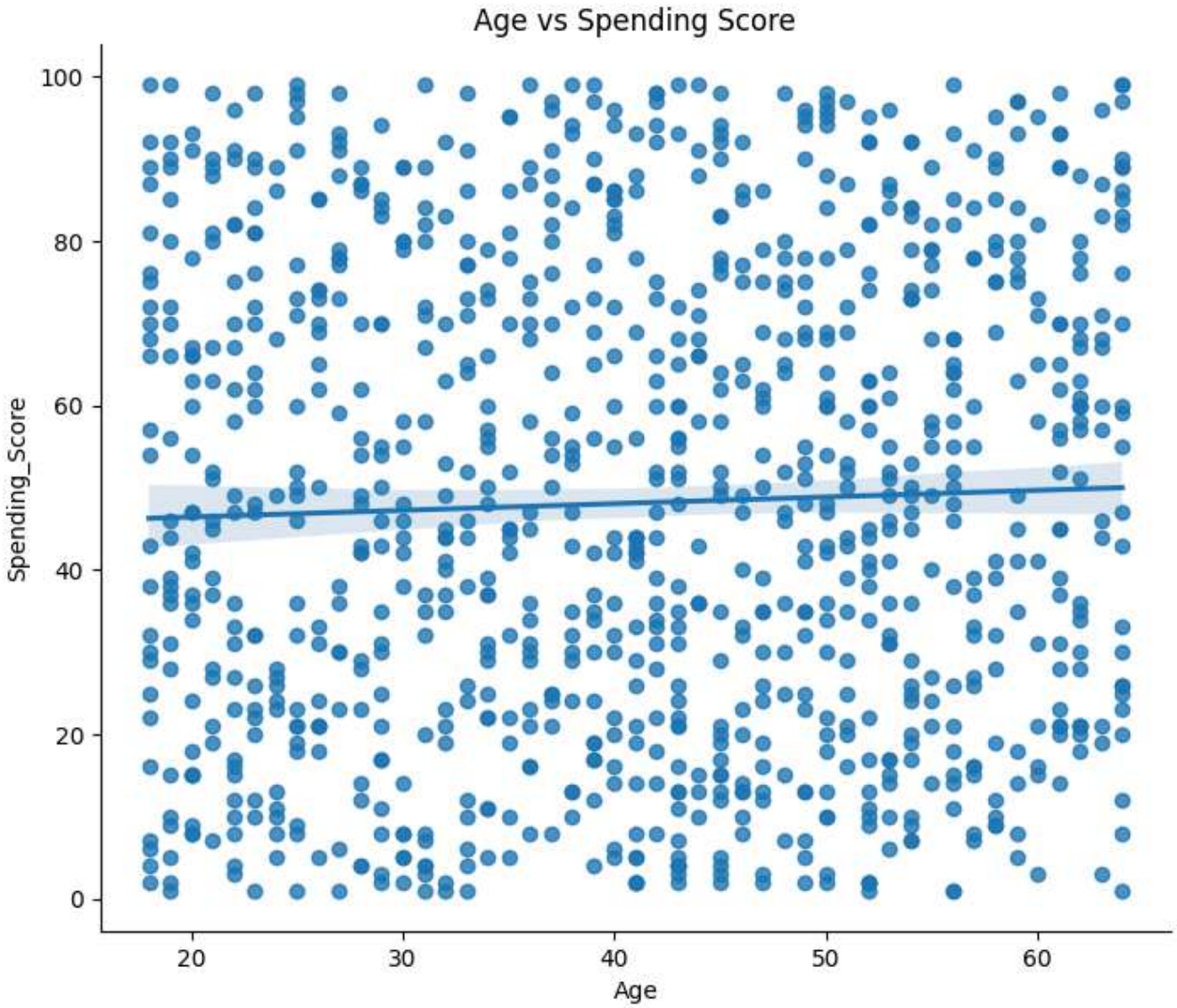
In [27]:

```
# 10. Compare purchasing behavior across Gender.

import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x='Gender', y='Spending_Score', data=df)
plt.title("Purchasing Behavior Across Gender")
plt.show()
```

In [32]:

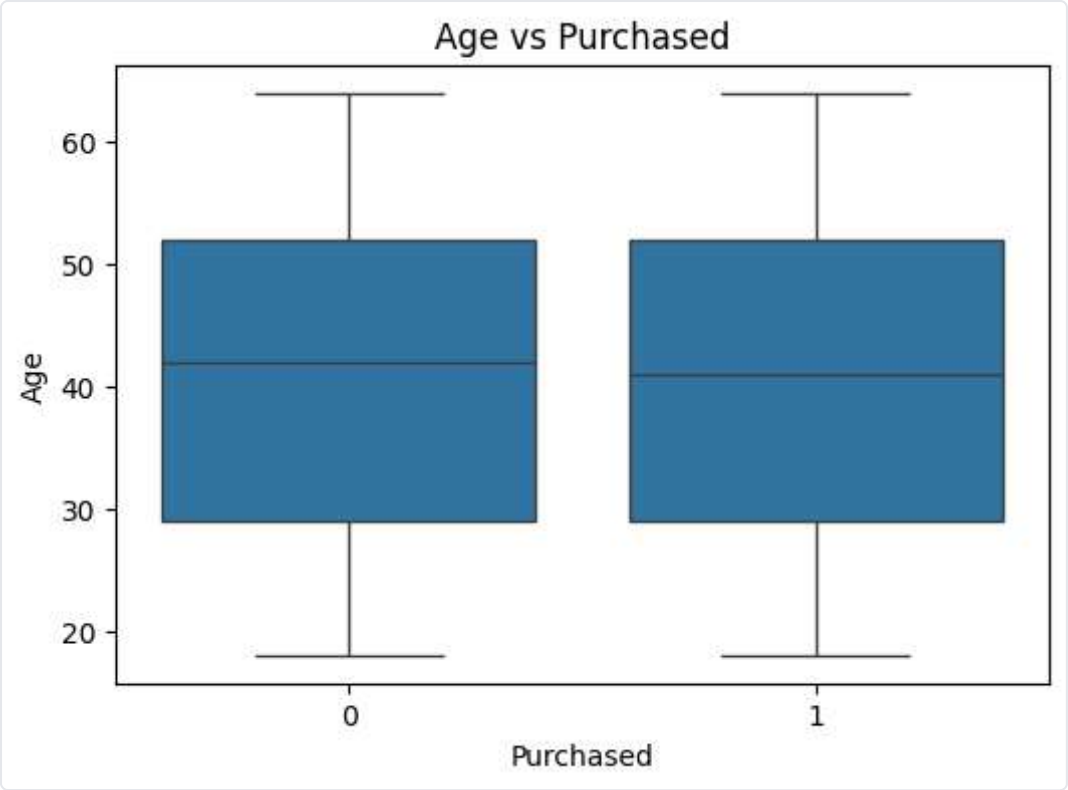
```
# 11. Does Age influence Spending Score?

import seaborn as sns
import matplotlib.pyplot as plt

# Correlation
corr = df['Age'].corr(df['Spending_Score'])
print(f"Correlation between Age and Spending Score: {corr:.2f}")

# Scatter plot with trend line
sns.lmplot(x='Age', y='Spending_Score', data=df, height=6, aspect=1.2)
plt.title("Age vs Spending Score")
plt.show()
```

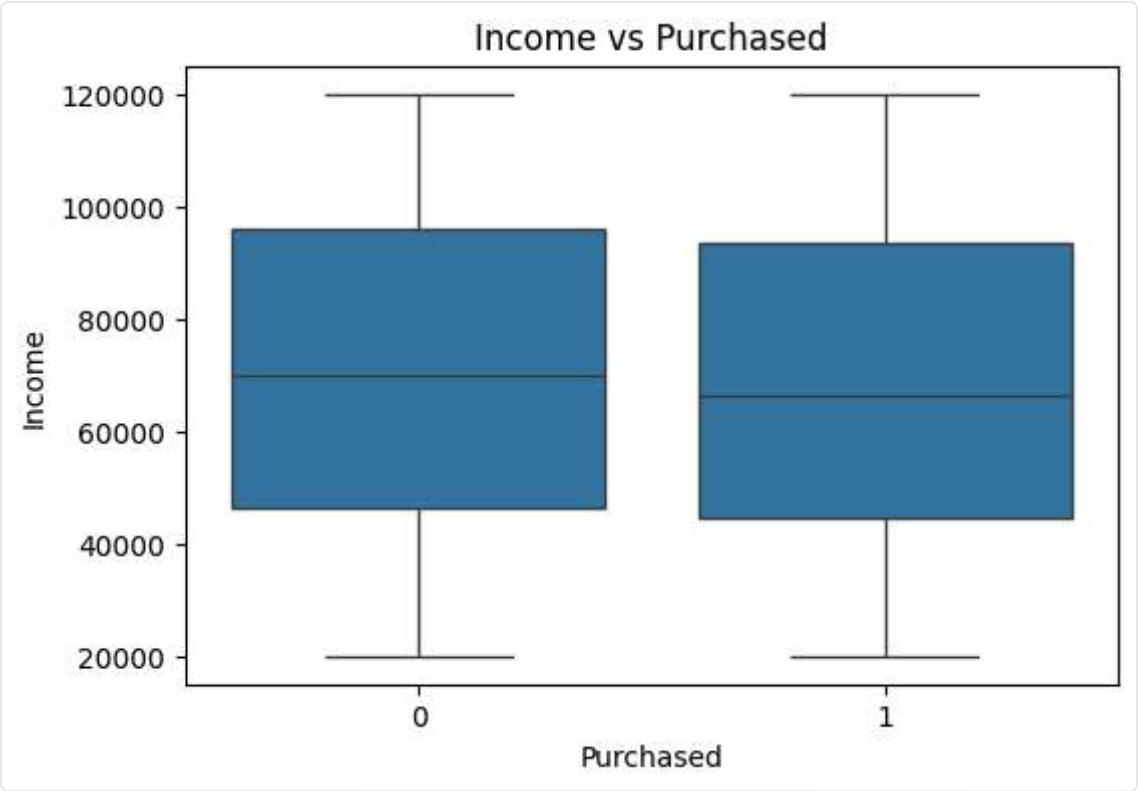
```
stdout:
Correlation between Age and Spending Score: 0.04
```

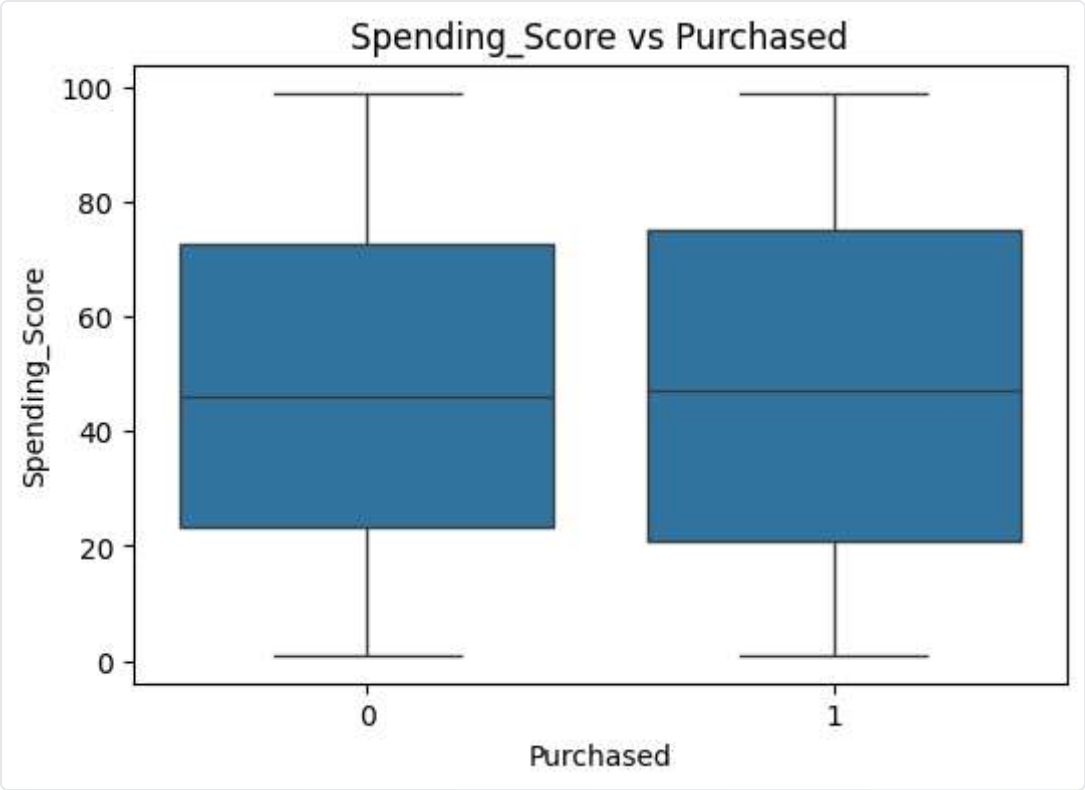


In [33]:

```
# 12. Analyze how multiple features affect Purchased.

import seaborn as sns
import matplotlib.pyplot as plt

numeric_features = ['Age', 'Income', 'Spending_Score']

for feature in numeric_features:
    plt.figure(figsize=(6,4))
    sns.boxplot(x='Purchased', y=feature, data=df)
    plt.title(f"{feature} vs Purchased")
    plt.show()
```


Age vs Purchased
Age
60
50
40
30
20
0
1
Purchased


Income vs Purchased
Income
120000
100000
80000
60000
40000
20000
0
1
Purchased



In [34]:

```
# 13. Create a new feature: Age Group (Young, Adult, Senior).

bins = [0, 25, 55, 100]
labels = ['Young', 'Adult', 'Senior']

df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

print(df[['Age', 'Age_Group']].head())
```

stdout:

```
   Age Age_Group
0   56    Senior
1   46     Adult
2   32     Adult
3   60    Senior
4   25     Young
```

In [35]:

```python
# 14. Convert Gender into numerical form (encoding).

# Map Male → 0, Female → 1 (or vice versa)
df['Gender_Num'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Check result
print(df[['Gender', 'Gender_Num']].head())
```

stdout:

```
   Gender  Gender_Num
0    Male           0
1  Female           1
2  Female           1
3  Female           1
4    Male           0
```

In [36]:

```python
# 15. Apply feature scaling (standardization or normalization).

from sklearn.preprocessing import StandardScaler

# Select numeric features to scale
numeric_features = ['Age', 'Income', 'Spending_Score']

scaler = StandardScaler()
df[numeric_features] = scaler.fit_transform(df[numeric_features])

# Check result
print(df.head())
```

```
stdout:
   Customer_ID       Age    Income  Spending_Score  Gender  Purchased  \
0            1  1.112882 -1.544373       -1.039376    Male          0
1            2  0.371652 -0.525705       -0.279901  Female          1
2            3 -0.666069 -1.431480       -0.935812  Female          0
3            4  1.409373 -1.682471       -1.142941  Female          0
4            5 -1.184929  0.478575        1.687830    Male          0

  Age_Group  Gender_Num
0    Senior           0
1     Adult           1
2     Adult           1
3    Senior           1
4     Young           0
```